

TED UNIVERSITY

CMPE492 Senior Project II

BioAuth Project Low-Level Design Report

16.10.2026

Team Members:

Esra Eda Kılıç - Computer Engineering
Enes Taşçı - Computer Engineering
Ömer Taylan Alemdar - Computer Engineering
Batuhan Mert Korkmaz - Computer Engineering

Supervisor:
Dr. Ulaş Güleç

Jury Members:
Prof. Dr. Saiful Islam
Prof. Dr. Tolga Kurtuluş Çapın

Contents

1. Introduction....4
1.1 Object Design Trade Offs....5
1.1.1 Research Contribution and Design Novelty....6
1.1.2 Implementation Choices for First Review....6
1.1.3 Accepted Design Decisions (ADR: Architecture Decision Record)....6
1.2 Interface Documentation Guidelines....7
1.2.1 Postconditions and Privacy....8
1.3 Engineering Standards....8
1.4 Definitions and Abbreviations....9
1.5 Requirements and Source Priority....10
2. Packages....10
2.1 Package Responsibilities and Design Patterns....12
2.1.1 Core Public Facade....12
2.1.2 Object relationships....12
3. Class Interfaces....13
3.1 Shared Types and Evidence Contracts....13
3.1.1 Status Interpretation....14
3.1.2 EvidenceNormalizer....14
3.2 Session Management and Lifecycle....14
3.2.1 SessionService....14
3.2.2 Session State Transitions....15
3.2.3 Initial Operational Defaults....15
3.3 Capture and Quality Assessment....16
3.3.1 CaptureBuffer....16
3.3.2 QualityAssessor and ROITracker....16
3.3.3 Capture Transport Profile....17
3.4 Face Verification and Reference Templates....17
3.4.1 FaceMatcher....17
3.4.2 Model Adapter Selection....18
3.4.3 TemplateService....18
3.5 Visual Presentation Attack Detection....19
3.5.1 VisualPADService....19
3.5.2 Baseline and Sampling....19
3.5.3 Attack Flags and Limits....19
3.5.4 Versioned Model Manifest....20
3.6 Physiological Signal Processing....20
3.6.1 RPPGPipeline....20
3.6.2 SignalProcessor Contract....20
3.6.3 FrequencyAnalyzer....21
3.6.4 Short-window Behavior....21
3.7 Fusion Strategies and Decision Policy....21
3.7.1 FusionStrategy....21

3.7.2 Common safety gate and precedence....22
3.7.3 L1 and Quality-only C-Q....22
3.7.4 ReliabilityAssessor and PatternAnalyzer....23
3.7.5 L2 Policy....23
3.7.6 L3 Advisory Model....23
3.7.7 Illustrative decisions without invented scores....24
3.8 Enrollment and Transaction Authorization....25
3.8.1 EnrollmentService and ReviewService....25
3.8.2 Enrollment State Transitions....25
3.8.3 Pending Review Evidence....26
3.8.4 TransactionGuard....26
3.9 Persistence and Data Lifecycle....27
3.9.1 Persistence Entities and Integrity Constraints....27
3.9.2 Initial Retention Profile For The Controlled Demo....28
3.10 REST Interfaces....29
3.10.1 Canonical REST Endpoints....29
3.10.2 Response and Error Contract....29
3.11 WebSocket and Web Client Interfaces....31
3.11.1 WebCaptureClient and EventGateway....31
3.11.2 Canonical Channels and Events....31
3.12 Security and Failure Handling....33
3.12.1 Roles and Permissions....33
3.12.2 Threat and Control Boundaries....33
3.12.3 Failure Behavior....33
3.13 Evaluation and Metric Interfaces....35
3.13.1 ExperimentRunner....35
3.13.2 Evaluation Configurations....35
3.13.3 MetricCalculator....36
3.14 Deployment and Performance Design....37
3.14.1 Deployment Architecture....37
3.14.2 Runtime Ownership and Limits....37
3.14.3 Performance Targets and Measurement....37
3.14.4 Release and Integration....38
3.15 Requirement Traceability and Verification....38
3.15.1 Requirement Traceability Matrix....38
3.15.2 Version 1 Review and Implementation Gates....39
4. Glossary....40
5. References....42

1. Introduction

BioAuth is a single-use biometric verification system used as a secondary level of authentication prior to performing high-risk digital tasks. BioAuth's core component is a Quality-and Attack-Aware Fusion Engine. This engine takes into consideration several variables including facial recognition results, visual presentation attack evidence, rPPG-derived physiological characteristics and image/video acquisition quality. The engine then uses this information to determine whether the output from each source is reliable and if there are any contradictions among those outputs. Based on this analysis, the engine makes a decision of ACCEPT, REJECT or RETRY and provides structured reasons for each outcome. The engine uses physiological signals as supporting data for determining if a user is physically present during the transaction; however, it does not use these signals as direct evidence of liveness. The consuming application remains responsible for account state and authorization of the requested action.

The representative integration is a demo platform that supports enrollment via document-portrait and live-face comparison, manual review and step-up verification before changing the payout destination of an active account. To activate the account, the enrollment attempt must achieve a successful biometric verification result and receive approval from an administrator. A RETRY result requires a new capture while a REJECT result prevents activation on that attempt; neither result can be overridden by routine administrator approval. The document portrait is used only as a reference for facial comparison, the system does not make any judgment about official document authenticity or provide full Know Your Customer (KYC) verification.

This level of design is based upon the requirements and high-level design established in CMPE 491, and further revised according to the BioAuth V2 reference, which involves the creation of a separate Visual PAD module, the assessment of evidence reliability/conflicts, the definition of comparative fusion strategies and a web-first approach. This report contains the description of the design, which requires the specification of the packages, classes, interfaces, and interactions leading to the implementation of the Quality- and Attack-Aware Fusion Engine as its core.

The initial implementation includes a reusable Python Core, a FastAPI service, a TypeScript capture SDK, React interface for users and administrators, PostgreSQL persistence and a controlled experiment runner. The experiment runner supports comparisons of fusion strategies using the same module outputs, including fixed-weight, quality-only and quality- and attack-aware approaches.

Dynamic light challenges, native secure capture, deep-learning based rPPG models, document OCR and NFC are deferred extensions. The initial delivery focusses on a mobile compatible web

application. The deferral of the native Android implementation changes the scope as described in earlier reports and is subject to review with the project supervisor.

1.1 Object Design Trade Offs

Choice	Rationale and consequence
Required rPPG gate	Retain physiology as an actual acceptance prerequisite. This can increase genuine-user retries; C-Q and L2 may not silently discard the channel to accept.
Classical CHROM and POS	Keep the established signal-processing baseline and expose both outputs for evaluation. No deep-rPPG training dependency is required for the first implementation.
Web-first capture	Reduce client integration effort. Browser capture does not prove hardware sensor origin; injection results are reported separately from physical presentation attacks.
Server-side inference	Keep model versions and decision rules under server control. Network transport, compression and queueing become part of the signal-quality and latency budget.
Modular core and application layer	Core accepts data and reference vectors through typed interfaces. It does not update accounts, perform payouts or depend on FastAPI or PostgreSQL.
Finite capture window	Initial development profile uses a five-second valid window. Short-window physiological estimates can be uncertain; failure to obtain usable evidence produces RETRY.
Encrypted template persistence	Retain a reference for future 1:1 verification while keeping ordinary session frames transient. Enrollment evidence needs a separate temporary review lifecycle.
Rule-first fusion	Implement auditable L1 and L2 before learned L3. L3 is evaluated offline until a versioned model and calibration profile meet the shared acceptance constraints.

These design principles are suitable for the first iteration because they maintain decision-making determinism, decouple biometric algorithms from delivery and persistence mechanisms, and make evidence and policy versions practically reproducible. Their operational adequacy will be

empirically evaluated using contract tests, subject-disjoint validation sets, latency/resource profiling, and controlled comparisons of the proposed fusion strategies.

1.1.1 Research Contribution and Design Novelty

The research hypothesis is that quality- and attack-aware fusion improves the trade-off between accept of attacks, rejection of legitimate inputs, abstention and latency, compared to both fixed and quality-only policies. The design does not presume that fusing face, PAD and rPPG is novel, or that one should not consider physiological signals spoofed. Replay and synthetic signals are explicit under the threat model.

L1, C-Q, L2 and L3 all share the same mandatory safety gate. The L2 fusion adds explicit channels reliability and suspicious pattern handling, and any improvement due to these additions are teased-out from the overall effect of a shared rPPG gate. By reporting the performance of the shared gate separately, we avoid implying that the better results from a more complex fusion algorithm.

The main design innovation is not the use of one biometric signal, but rather using explicit slots for measurement status, measurement reliability and attack-indications, allowing to version the fusion contract. This enables proper comparison of C, C-Q and L2 policies while maintaining the shared safety gates.

1.1.2 Implementation Choices for First Review

The initial transfer, five-second window, buffer limits, and expiry periods below are engineering defaults for the implementation and profile, not measured abilities or biometric thresholds. They are stored in versioned configuration. For changes that affect requirements or experiment comparability a recorded design revision is required.

1.1.3 Accepted Design Decisions (*ADR*: *Architecture Decision Record*)

ID	Decision
ADR-01	Every channel separates measurement status from its numeric result. Missing measurements use null rather than zero.
ADR-02	Account activation requires both successful biometric enrollment and an authorized manual approval. Review cannot override biometric failure.

ADR-03	Add C-Q, a quality-only fusion baseline, between fixed fusion C and quality/attack-aware D.
ADR-04	Report genuine and attack outcome distributions, coverage, latency and technical failures together.
ADR-05	Usable rPPG is mandatory for acceptance in L1, C-Q, L2 and L3. Missing or unreliable rPPG leads to RETRY unless a reliable rejection reason exists.
ADR-06	The reference template is derived from the successful camera capture and activated only after administrator approval.

Version 1 specifies intended behavior. Threshold values, measured throughput and model accuracy will be established by implementation and validation; illustrative values in earlier material are not adopted as empirical results.

1.2 Interface Documentation Guidelines

Python public interfaces make use of type annotations and explicit return types of results. Boundary request/response bodies use Pydantic schemas, and the web client makes use of matching TypeScript types. Internal units are seconds for capture time, milliseconds for elapsed processing durations, hertz for frequencies, BPM for pulse estimates, and decibels for SNR. Server timestamps are UTC ISO 8601, and capture elapsed timestamps are monotonic offsets.

Rule	Interface requirement
Naming	Python methods use snake_case. Class and DTO names use PascalCase. JSON fields use snake_case. Event types use dotted lowercase names.
Identity	session_id and event_id are UUIDs. user_ref is a platform-scoped pseudonymous reference. Client-supplied IDs never confer access.
Score semantics	Each score declares its direction and range. Face cosine similarity and PAD bona-fide score increase toward a favorable result. Raw SNR remains in dB.

Rule	Interface requirement
Missing values	Nullable result fields are null when unavailable. A zero value is preserved only when it is an actual computed measurement.
Probability claims	Raw model scores are not probabilities. calibrated_score is nullable and includes a calibration profile version when present.
Errors	Expected measurement failures return typed status and reason codes. Unexpected exceptions become sanitized technical failures at the application boundary.
Mutability	EvidenceSnapshot and Decision are immutable. Session changes use atomic state/version checks; a finalized snapshot cannot be replaced.
Versioning	Requests declare schema_version. A major incompatible version uses a new /vN route. A decision records model, policy, calibration and capture profile versions.

1.2.1 Postconditions and Privacy

Consider that every public method documented preconditions, output guarantees, failures and side effects. A successful service call does not imply successful authentication. VALID describes whether evidence is interpretable, not whether it supports acceptance. APIs do not expose embeddings, raw frames or hidden thresholds in ordinary user responses.

A decision references an immutable snapshot and reason codes generated by the executed rules. Reasons are ordered by decision priority. Internal diagnostic reasons may be richer than user guidance. Admin report serialization filters fields by role and session mode before transmission.

1.3 Engineering Standards

UML 2.5.1 provides the notation for class interfaces, realization, dependency and state models. Package and class view document ownership and interactions; transition tables specify lifecycle guards. The object-design approach follows Bruegge and Dutoit: refine analysis objects, introduce solution objects and specify subsystem interfaces. [6, 7]

ISO/IEC 30107-3:2023 is the reference for presentation-attack evaluation terminology and the later Test Plan. This design does not assert certification or full conformance. HTTP, JSON, OpenAPI and encrypted HTTPS/WSS provide interoperable service boundaries; library/framework use alone is not a standards-compliance claim. [9, 10]

1.4 Definitions and Abbreviations

Term	Meaning in this design
Verification	One 1:1 comparison against an existing reference combined with presentation and physiological evidence.
Enrollment	Creation of a candidate reference from a checked live camera capture; activation follows review.
PAD	Presentation Attack Detection. Visual PAD analyzes image evidence of a bona-fide or attack presentation.
rPPG	Remote photoplethysmography; extraction of pulse-related optical changes from facial video.
ROI	Region of interest: forehead, left cheek or right cheek pixels used for RGB traces.
CHROM / POS	Two classical rPPG extraction strategies evaluated from the same temporally sampled RGB data.
Evidence	A versioned channel result with status, quality, origin and nullable measurement fields.
Reliability	The defined suitability of a measurement under the observed capture conditions; not identity confidence.
Abstention	RETRY: insufficient usable evidence for a definitive biometric decision.
Template	Encrypted facial embedding from an approved enrollment capture, associated with its embedding-model version.
Coverage	Fraction of attempts that produce a definitive ACCEPT or REJECT; technical failures are not silently omitted.
SDK / API	Client integration library / service interface. The SDK does not make the authoritative security decision.
L1 / L2 / L3	Alternative fixed-rule, quality/attack-aware and ML-assisted decision strategies, not mandatory sequential stages.

1.5 Requirements and Source Priority

Requirements analysis for the analysis requirements FR-01 to FR-10 remain as traceability anchors. V2-FR-11 to V2-FR-17 mark additions from the V2 reference. The accepted decisions in section 1 clarify ambiguous behavior. Historical wording that treated rPPG as evidence of freshness is being replaced with explicit evidence and threat limits. [3-5]

Operational biometric verification and controlled ablation are different execution modes. A and B baselines deliberately omit channels that cannot authorize demo transactions. Only a versioned deployment policy that enforces all required gates can generate an actionable ACCEPT.

2. Packages

Dependency direction keeps domain algorithms independent of delivery frameworks. Application services call Core through typed contracts and use repository interfaces for persistence. The experiment runner calls the same Core components without HTTP. Figure 1 shows package dependencies; dashed arrows indicate use.

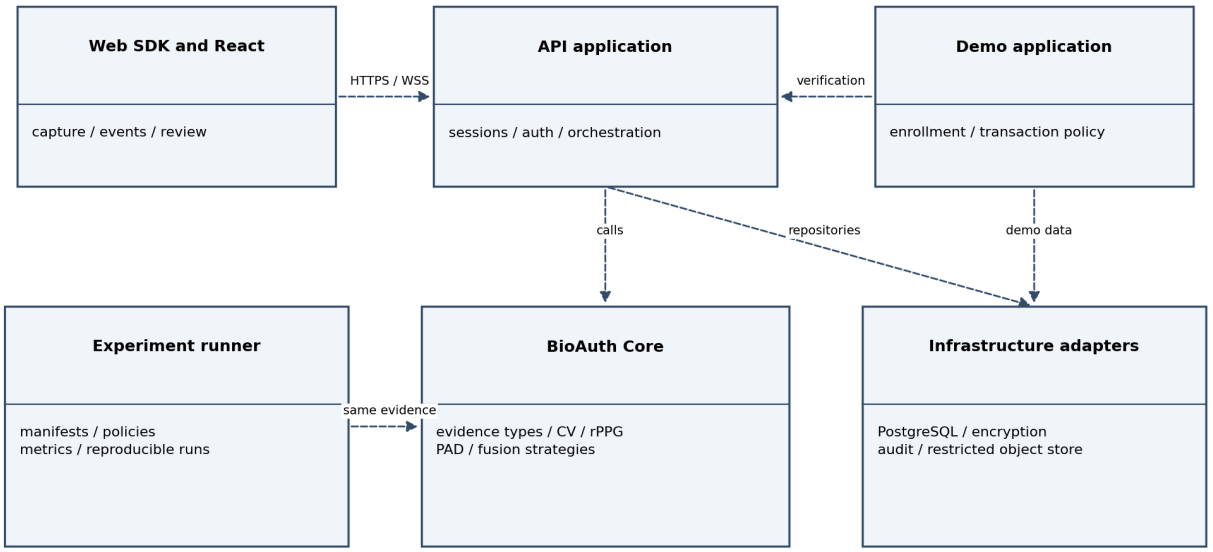


Figure 1. Package dependency view for the first implementation

Package	Responsibilities and exports
bioauth_core.contracts	Enums, Evidence, EvidenceSnapshot, Decision, CaptureProfile and PolicyProfile.
bioauth_core.capture	DecodedFrame, timestamp validation, ROI traces and capture/quality utilities.
bioauth_core.face / pad	FaceMatcher and PADAdapter protocols; model-specific adapters.
bioauth_core.physiology	ROITracker, CHROMExtractor, POSExtractor, SignalProcessor and SignalQualityAssessor.
bioauth_core.fusion	SafetyGate, EvidenceNormalizer, ReliabilityAssessor, PatternAnalyzer and strategy implementations.
apps.api	SessionService, pipeline orchestration, REST, WSS, authentication and admission control.
apps.demo	EnrollmentService, ReviewService and TransactionGuard; application account policy.
infra / experiments	Repositories, encryption and audit adapters / manifests, experiment execution and metrics.
packages.web / apps.web	TypeScript SDK / React user portal and authorized admin views.

2.1 Package Responsibilities and Design Patterns

2.1.1 Core Public Facade

A selected strategy, and feature extraction, are coordinated by BioAuthEngine.verify(capture: CaptureBatch, reference: ReferenceVector, policy: PolicyProfile) -> VerificationResult. This does not persist any data or perform any business logic; it returns an EvidenceSnapshot and Decision. The use of multiple policies against the same evidence is supported via a different extract_evidence(...) function. CaptureBatch contains measured time and decoded RGB frames, rather than HTTP objects.

2.1.2 Object relationships

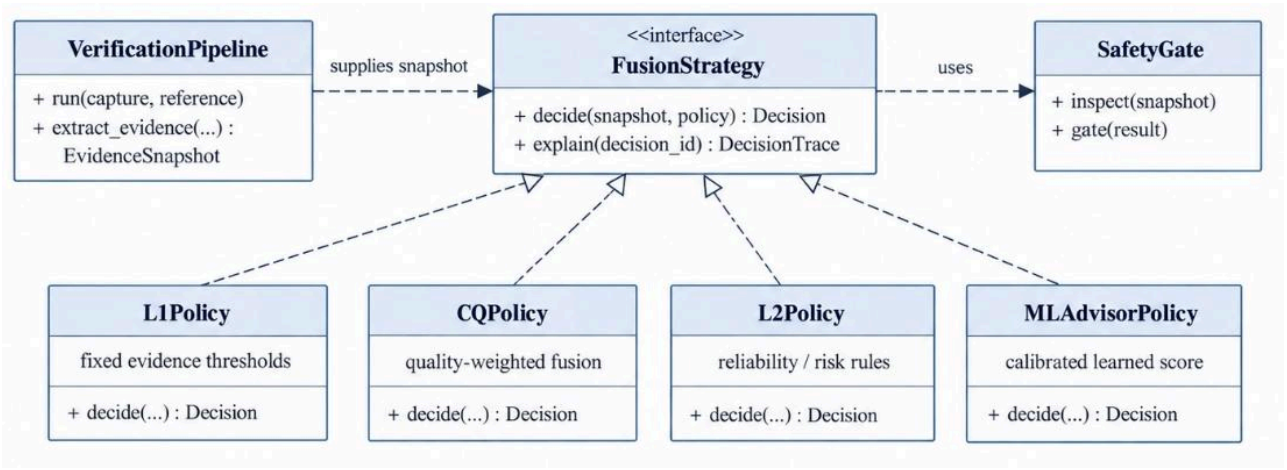


Figure 2. Fusion interface realization and dependencies. Hollow arrowheads point to the realized interface; dashed arrows indicate use.

Pattern	Application and reason
Strategy	CHROM/POS extractors and L1/C-Q/L2/L3 strategies can be exchanged without changing orchestration.
Adapter	ArcFace and PAD backends translate external outputs into stable domain contracts; model-specific preprocessing stays inside the adapter.
Repository	Session, template and audit persistence are accessed through interfaces. Core has no SQL dependency.
State machine	SessionService enforces lifecycle transitions and prevents writes after finalization.
Observer / event publication	Decisions are not made by clients; instead, application services report filtered domain events for SDK and admin progress views.

One capture buffer and one final proof image belong to each session. One face result, one PAD result, one rPPG result, and one capture-quality result are all included in the snapshot. Both extractor diagnostics are present in a rPPG result. Only one template version is active for that model profile at a time, and ReferenceTemplate is a part of a single scoped user reference.

3. Class Interfaces

3.1 Shared Types and Evidence Contracts

Type	Required fields and constraints
EvidenceStatus	VALID | INSUFFICIENT_DATA | UNRELIABLE | ERROR.
Evidence<T>	evidence_id, session_id, channel, status, value:T|null, quality:QualityVector, reliability:float|null, reason_codes[], captured_range, model_version, schema_version.
FaceValue	cosine_similarity in [-1,1], matched:bool, reference_version. Comparison is valid only with compatible model and preprocessing versions.
PADValue	bona_fide_score in [0,1], attack_flags[], aggregation statistics, sampled_frame_count; calibrated_score and calibration_version may be null.
RPPGValue	bpm_estimate:float|null, snr_db:float|null, usable:bool, duration_s, observed_fps, per_roi metrics, POS/CHROM agreement diagnostics.
EvidenceSnapshot	snapshot_id, session_id, face, pad, rppg, capture_quality, session_checks, configuration_hash and created_at. Immutable after seal().
Decision	decision_id, session_id, result, reason_codes[], policy_version, snapshot_id, processing_ms, uncertainty:float|null, created_at.
PolicyProfile	policy_id, version, required_channels, gate settings, thresholds, normalization and calibration versions, model allowlist and configuration_hash.

3.1.1 Status Interpretation

VALID indicates that a measurement can support rejection and be utilized by a decision rule. INSUFFICIENT_DATA denotes an incomplete temporal window or an insufficient number of acceptable frames. UNRELIABLE refers to measures that are present but the specified quality checks are not successful. A technical processing failure is indicated by an error. Values that are not available are null. A gate cannot be satisfied by an unreliable diagnostic value, which can only be kept in limited research output.

Acceptance of RPPG requires usable=true and status=VALID. Face and PAD need to be in VALID status in addition to their policy requirements. Although it stops acceptance, technical errors are not considered rejections of biometric attacks. When another channel is unavailable, REJECT may still be supported by an independently reliable attack or mismatch.

3.1.2 EvidenceNormalizer

State and dependencies: Versioned channel transformations; no mutable per-user state.

Operation	Contract
normalize(e: Evidence, profile) -> NormalizedEvidence	Preserve raw values and status. Convert usable metrics to declared [0,1] desirability values using validation-fitted mappings.
validate(snapshot) -> ValidationReport	Check ranges, finite values, channel completeness, matching session IDs and model versions.

Invariants and behavior: Never substitute zero for missing values. Refuse NaN/Infinity near the edge. Without a clearly chosen strategy, do not average identification, measurement quality, and assault evidence.

3.2 Session Management and Lifecycle

3.2.1 SessionService

State and dependencies: SessionRepository, PolicyRegistry, Clock, EventPublisher; session version, expiry and nonce hash.

Operation	Contract
create(actor, intent, user_ref, action_id, mode) -> SessionTicket	Authorize reference and action access; bind immutable intent, action digest, policy and expiry. Return short-lived session credentials.

Operation	Contract
start_capture(id, actor) -> SessionView	Require CREATED state, consent and valid session. Allocate bounded buffer and move to CAPTURING.
finalize(id, actor, idempotency_key) -> FinalizeReceipt	Seal accepted frames once; start processing. Repeated identical calls return the same job/decision reference.
cancel(id, actor) -> SessionView	Close an unfinished session and release capture resources.
complete(id, decision, expected_version) -> SessionView	Atomically persist immutable decision and mark COMPLETED; reject stale competing writes.

Behavior and invariants: Nonce/session binding protects request reuse, not physical camera origin. No other session may append frames or consume a result. Finalized frames and policy versions cannot be changed.

3.2.2 Session State Transitions

From	Trigger and guard	To
CREATED	Consent, ownership and expiry checks pass	CAPTURING
CAPTURING	Finalize freezes the capture batch	PROCESSING
PROCESSING	Evidence extraction completes	EVALUATING
EVALUATING	Decision and audit commit together	COMPLETED
Any unfinished state	User cancellation	CANCELLED
Any unfinished state	Server deadline exceeded	EXPIRED
Any unfinished state	Unrecoverable infrastructure failure	FAILED

A different result for COMPLETED is either ACCEPT, REJECT, or RETRY. The lifetime outcomes CANCELED, EXPIRED, and FAILED have an explanation code and a null biometric decision. They never give permission. A simple timeout is not proof of spoofing; reuse or manipulation is a security rejection event. This distinguishes between biometric statistics and technical failure.

3.2.3 Initial Operational Defaults

The capture is restricted to 15 seconds of elapsed acquisition with a single five-second valid analysis window; the session TTL is 120 seconds. A RETRY initiates a fresh session associated with the same attempt group. For a single user or activity, the demo allows up to three capture attempts in ten minutes before blocking more attempts until a platform-controlled restart. Usability and load validation are necessary for these customizable values.

3.3 Capture and Quality Assessment

3.3.1 CaptureBuffer

State and dependencies: Per-session bounded ring of decoded frames and capture/receive timestamps; no disk writes.

Operation	Contract
append(packet: FramePacket) -> FrameReceipt	Check sequence, dimensions, format and timing; decode once and attach server receive time.
seal(profile) -> CaptureBatch	Select a contiguous acceptable window. Preserve gaps and observed FPS; report insufficient data rather than fabricating frames.
release() -> None	Drop frame references on completion, cancellation, expiry or error.

Behavior and invariants: No cross-session buffers. Duplicate or decreasing sequence numbers are rejected. Missing frames are recorded, not silently hidden.

3.3.2 QualityAssessor and ROITracker

State and dependencies: Landmark adapter, ROI map version and capture quality profile.

Operation	Contract
assess_frame(frame, landmarks) -> FrameQuality	Measure face count, blur, exposure, pose, coverage and motion. No face/multiple faces invalidate the capture interval.
track(frame, previous) -> ROIResult	Produce forehead and cheek polygons from normalized landmarks. Exclude eyes/mouth and invalidate out-of-frame or too-small regions.
summarize(frames, timing) -> QualityVector	Return usable duration, measured FPS, gap statistics, exposure/motion stability and channel-specific eligibility.

Behavior and invariants: Quality failure is not itself attack evidence. A change of face track invalidates the current window. Face and PAD may use sampled frames; RGB traces use every retained valid frame with timing.

3.3.3 Capture Transport Profile

The first integration targets 30 frames per second using binary WSS frames encoded as JPEG with a maximum resolution of 640 by 480 and a specified quality of 95. The exact format and decoded dimensions are verified by the server. Before this profile is qualified, a research capture adapter measures transport-induced signal degradation by accepting lossless frames. JPEG settings do not ensure rPPG integrity; they are merely a prototype option.

A frame packet contains encoded image bytes, a UTF-8 JSON header, and a four-byte network-order header length. Schema_version, session_id, seq, capture_offset_ms, width, height, and encoding are the header fields. The maximum packet size is 512 KiB, with a header restriction of 4 KiB. There are a maximum of 60 encoded packets in the first inbound queue and 180 frames in the decoded analysis buffer. Overflow does not allow a queue to grow forever; instead, it records a gap and issues CAPTURE_BACKPRESSURE.

When the SDK's local transmit budget is exceeded, it drops a newly generated frame and retains a maximum of one pending encoded frame. Each drop is tallied. The window's usability is determined by measured gaps and timestamp validity rather than the intended FPS.

3.4 Face Verification and Reference Templates

3.4.1 FaceMatcher

State and dependencies: EmbeddingAdapter, alignment profile, reference version and calibrated threshold profile.

Operation	Contract
embed(frame: RGBFrame, face: FaceRegion) -> EmbeddingResult	Align a quality-approved single face; normalize the adapter output. Return ERROR if model loading/inference fails.
compare(probe, reference, profile) -> Evidence<FaceValue>	Require equal embedding model, dimension and preprocessing versions. Compute cosine similarity and the configured match decision.
select_probe(batch, quality) -> ProbeSelection	Choose the best eligible frame by deterministic quality ordering; do not pick the frame with the highest identity score.

Behavior and invariants: A valid mismatch can cause REJECT. Poor-quality or missing face input produces RETRY when no independent rejection exists. Raw face scores are not described as calibrated identity probabilities.

3.4.2 Model Adapter Selection

DeepFace is a potential wrapper rather than a distinct comparable model; the implementation baseline employs an ArcFace-family embedding via a changeable adapter. Before a profile is activated, a ModelManifest must specify the precise architecture, weight checksum, preprocessing, vector dimension, and allowed use. Vectors from different models are not compared; switching model versions necessitates compatible enrollment or a new template. [5]

3.4.3 TemplateService

State and dependencies: TemplateRepository and encryption provider; scoped user identity and consent record.

Operation	Contract
stage(enrollment_id, capture, manifest) -> CandidateTemplate	Create an encrypted candidate from a quality-approved camera frame only after successful enrollment evidence.
activate(candidate_id, review_id) -> ReferenceTemplate	Require a valid approval bound to the same candidate and snapshot; atomically activate the version.
load(user_ref, actor, model_version) -> ReferenceVector	Authorize scope, status and expiry; decrypt only within the inference process.
revoke(template_id, reason) -> None	Mark unusable and schedule key/ciphertext deletion according to the lifecycle policy.

Behavior and invariants: The uploaded document portrait is not used as a future reference embedding, but rather for enrollment comparison. Returning-user verification cannot be authorized by candidate templates. Embeddings are never brought back to regular admin views or the browser.

Vector search and pgvector are not needed for the 1:1 use case. The current version and authorized user_ref are used to retrieve templates. That reference is provided by the demo program; arbitrary user identification or enumeration is outside the purview.

3.5 Visual Presentation Attack Detection

3.5.1 VisualPADService

State and dependencies: PADAdapter, crop profile, sampling policy and calibration registry.

Operation	Contract
prepare(frame, face_region) -> PADInput	Apply the exact model crop, color order, scale and normalization from the manifest. Preserve context required by that model.
infer(inputs) -> FramePADResults	Return bona-fide scores with inference timing. A failed inference is ERROR, not an attack score.
aggregate(results, quality) -> Evidence<PADValue>	Use deterministic evenly spaced eligible frames; record median, lower quantile, spread and count.
qualify(result, profile) -> PADAssessment	Map score bands to benign, uncertain or strong-attack evidence under the approved validation profile.

Behavior and invariants: Only a qualified score with usable PAD input can supply a strong-attack veto. A low score from an unusable crop does not automatically mean attack. Missing PAD evidence cannot be bypassed by strong face or rPPG values.

3.5.2 Baseline and Sampling

The first contender is MiniFASNet. Before integration, the adapter's documented preprocessing and licensed weights must be verified. Throughout the five-second analysis window, the first compute budget uniformly samples up to 15 frames. The validation profile includes the minimum eligible sample count and score thresholds; until that profile is established, the service is not eligible.

The impact of a single frame is limited by a median score. Inconsistent video behavior is revealed to L2 via a lower quantile and score spread. Uncertainty or evidence of an unobserved attack is not automatically calibrated in these figures. Held-out sessions must be used to verify temporal aggregation and model quality. Unsupported heatmaps are not generated merely for presentation.

3.5.3 Attack Flags and Limits

Instead of a particular label like replay or deepfake, a binary model supports a bona-fide/attack score. Only when an assault species is identified by a dedicated detector or a recognized experimental label can attack_flags do so. Labels from ground-truth experiments are never used as predictions in fusion.

Synthetic video, print, and screen replay are assessed independently. It's possible that a print/replay-trained model won't translate to face swaps or digital injection. The latter is still a different category of experiment. A dependable PAD rejection is not canceled by a positive rPPG signal. [5, 8]

3.5.4 Versioned Model Manifest

Model_id, architecture, artifact_sha256, runtime_version, crop_profile, color_order, score_direction, output_labels, calibration_id, and license_record are all necessary fields. Verification sessions cannot be accepted due to missing or mismatched manifests that fail service readiness.

3.6 Physiological Signal Processing

3.6.1 RPPGPipeline

State and dependencies: ROI RGB traces, CHROM/POS strategies, SignalProcessor and SignalQualityAssessor.

Operation	Contract
extract_rgb(batch, roi_map) -> RGBTraces	Compute spatial RGB means for each accepted ROI and retain acquisition offsets and invalid-sample masks.
estimate(traces, profile) -> RPPGAnalysis	Validate sampling, resample only within allowed small gaps, execute CHROM and POS and retain per-ROI diagnostics.
assess(analysis, quality) -> Evidence<RPPGValue>	Evaluate window duration, SNR, peak concentration, ROI coherence and method agreement. Set usable only when all required profile checks pass.

Behavior and invariants: A periodic peak is physiological evidence, not proof of fresh capture. Low SNR or absent pulse under poor conditions produces RETRY, not automatic spoof rejection. Both extraction methods are implemented; POS is the selected primary estimator and CHROM is a consistency/benchmark channel.

3.6.2 SignalProcessor Contract

process(rgb: float[T,3], times: float[T], config) -> SignalResult first validates finite channel means and timestamps. It normalizes traces according to the selected CHROM/POS method, derives the

pulse component, removes residual trend and applies a fourth-order Butterworth bandpass with 0.7–4.0 Hz cutoffs using second-order sections. Completed-window forward/backward filtering is used; the minimum input length and padding constraints are checked before execution. It is not described as a causal per-frame filter. [11–13]

3.6.3 FrequencyAnalyzer

Frequency Analyzer.Welch PSD with a Hann window is used in analyze(signal, fs, config) -> SpectrumResult; the first five-second profile uses a single full-length segment to prevent further lowering frequency resolution. FFT diagnostics are not required. BPM=60f is the dominating eligible peak. Zero padding does not add measurement information, although it may interpolate the spectrum.

Reproducibly, SNR is defined as 10 log10(Psignal/Pnoise), where Pnoise is the residual in-band power and Psignal is the sum of PSD in the cardiac band within one natural frequency bin of the chosen peak and, when in band, its second harmonic. Metrics are unavailable due to degenerate power and missing peaks. This is not a general clinical SNR standard; rather, it is an internal quality definition. Its thresholds and band width are set in stone along with the profile.

3.6.4 Short-window Behavior

Natural frequency spacing is roughly 0.2 Hz, or 12 BPM, at five seconds. Only when the signal is qualified is BPM displayed. POS/CHROM peak disagreement and normalized cross-ROI spectral agreement are noted; validation data is used to fit precise qualification boundaries. A plausible-looking 0 BPM is never achieved due to capture corruption, significant gaps, or insufficient duration. When the UI is unusable, it receives bpm_estimate=null along with a correction explanation.3.7 Decision policy and fusion tactics

3.7 Fusion Strategies and Decision Policy

3.7.1 FusionStrategy

State and dependencies: Immutable EvidenceSnapshot and a versioned PolicyProfile.

Operation	Contract
decide(snapshot, profile) -> Decision	Apply common gates before strategy-specific scoring; return a reproducible outcome and ordered reasons.
explain(decision_id) -> DecisionTrace	Expose fired rule IDs, eligible evidence references and profile version to authorized audit consumers.

Behavior and invariants: L1, C-Q, L2 and L3 are alternatives evaluated on the same evidence. In deployment exactly one approved policy is selected server-side. The client cannot choose a weaker strategy.

3.7.2 Common safety gate and precedence

Order	Condition	Outcome
1	Snapshot/session tampering, known replay of authorization, incompatible binding	REJECT; SESSION_INTEGRITY_FAILURE
2	VALID face evidence shows mismatch, or VALID qualified PAD supplies strong attack evidence	REJECT; FACE_MISMATCH or PAD_ATTACK_EVIDENCE
3	Required model has ERROR, with no independent reliable rejection	No acceptance; session FAILED with technical reason
4	Face, PAD, capture quality or rPPG is missing/unreliable; rPPG.usable is false	RETRY with channel-specific reason
5	All gates pass	Evaluate selected strategy and its acceptance/abstention bands

An expired or cancelled session is not evaluated and cannot be consumed. The ordering allows reliable rejection despite another missing channel while preventing poor input from being misclassified as mismatch. The physiological gate is required even when face and PAD are favorable.

3.7.3 L1 and Quality-only C-Q

Following conventional gates, L1 uses versioned normalization to create z=(face support, PAD support, physiological plausibility). With nonnegative weights adding up to one, its fixed score is S=sum(w_i z_i). Based on validation data, weights and a two-threshold decision band are chosen: S>=t_accept accepts, S<=t_reject rejects with POLICY_LOW_SUPPORT, otherwise RETRY. Prior to calibration, no numerical biometric criteria are created.

C-Q employs S_Q=sum(w_i q_i z_i)/sum(w_i q_i), where q_i is channel-specific measurement reliability, while maintaining the same features and common gates. RETRY is caused by a zero denominator. A near-zero weight cannot covertly eliminate missing physiology because the

required rPPG gate is applied prior to weighting. There are no extra attack-pattern rules used by C-Q. The same held-out methodology and search budget are used for all threshold selection.

3.7.4 ReliabilityAssessor and PatternAnalyzer

ReliabilityAssessor.evaluate (profile, snapshot) -> ReliabilityVector uses validation-fitted monotonic mappings of their quality inputs to yield distinct face, PAD, and rPPG reliability. Alignment, blur, and posture are used in face quality; crop eligibility and sample stability are used in PAD quality; duration, gaps, SNR, and ROI agreement are used in rPPG. Suitability, not the likelihood of legitimacy, is what reliability describes.

PatternAnalyzer.inspect(snapshot, profile) -> PatternFindings applies named, testable rules. In v1: PAD_TEMPORAL_INSTABILITY flags excessive score spread; PHYSIOLOGY_DISAGREEMENT flags excessive POS/CHROM or cross-ROI disagreement; PAD_SUSPECT_WITH_PULSE flags moderate qualified PAD suspicion despite usable pulse. A face match plus an attack is a possible replay scenario, not a physical contradiction. Findings record supporting evidence and rule versions.

3.7.5 L2 Policy

L2 uses the common gate first. Unless there is already a strong common rejection, L2 returns RETRY with the pertinent code if another validated suspicious pattern fires. If not, it uses preset decision bands and the quality-weighted score. A strong pulse cannot make up for a suspicion of PAD. L2 is distinguished from C-Q by its explicit conservative pattern layer; its cost in coverage has to be quantified. After reading the final test labels, additional rules cannot be introduced and must be validated in writing.

3.7.6 L3 Advisory Model

Regularized Logistic Regression is the first step in MLAdvisor.predict(features, manifest) -> RiskScore; XGBoost is an optional comparator. Face/PAD normalized scores, rPPG quality and agreement, capture quality, missingness indications, and pattern finds are all included in the feature vector. Excluded are test membership, ground-truth attack labels, and person IDs. Only training data is used for preprocessing.

L2 non-overridable safety holds and the shared gate are used by L3. The remaining aggregate-score decision is replaced by its learnt score. Validation data is used for calibration and decision criteria; if a model is absent or incompatible, L3 is not available instead of pretending that an L2 result is L3. Until L3 is specifically qualified, the deployed default stays L2.

3.7.7 Illustrative decisions without invented scores

Evidence	Decision and reason
Identity match; benign PAD; usable rPPG; stable capture; score supports acceptance	ACCEPT / EVIDENCE_SUFFICIENT
Identity match; benign PAD; unusable rPPG	RETRY / INSUFFICIENT_RPPG_QUALITY
Valid identity mismatch; otherwise favorable evidence	REJECT / FACE_MISMATCH
Strong qualified PAD attack; usable pulse	REJECT / PAD_ATTACK_EVIDENCE
Moderate PAD suspicion and usable pulse; L2 pattern fires	RETRY / PAD_SUSPECT_WITH_PULSE
PAD runtime failure and no reliable rejection	FAILED lifecycle / MODEL_PROCESSING_ERROR

3.8 Enrollment and Transaction Authorization

3.8.1 EnrollmentService and ReviewService

State and dependencies: Demo repositories, verification service, restricted review store and TemplateService.

Operation	Contract
submit_document(actor, file, consent) -> DocumentRef	Validate type/size and consent; isolate upload; extract one portrait. Return DOCUMENT_FACE_NOT_FOUND when unavailable.
verify_enrollment(id, capture) -> EnrollmentResult	Compare camera probe to document portrait and run PAD/rPPG/quality. Bind result to this document, candidate and capture.
request_review(id) -> ReviewCase	Only successful biometric enrollment can enter PENDING_REVIEW. Store minimal authorized review evidence.
review(id, reviewer, verdict, reason, expected_version) -> ReviewResult	Require reviewer role. Approve only current successful evidence and an unexpired candidate; rejection or recapture never activates a template.

3.8.2 Enrollment State Transitions

Behavior and invariants: No ordinary administrator override of failed biometrics. ACCEPT plus APPROVE are both required. A document portrait match is not official document authenticity. ReviewCase and account state are separate from the verification session state.

Enrollment state	Transition
DRAFT	Document and consent accepted -> CAPTURE_REQUIRED
CAPTURE_REQUIRED	Biometric ACCEPT -> PENDING_REVIEW; RETRY remains capture required; REJECT closes this attempt
PENDING_REVIEW	APPROVE -> ACTIVE; REQUEST_RECAPTURE -> CAPTURE_REQUIRED; REJECT -> REJECTED
ACTIVE	Template revocation or administrative suspension disables new authorizations

3.8.3 Pending Review Evidence

Approved access to a document portrait and a chosen camera still is required for manual evaluation. Instead of being an uncontrolled test-video archive, they are encrypted and kept in a different temporary review store with a clear function and preservation. A review is dependent on the precise candidate and snapshot. Earlier approval eligibility is nullified by recapture.

3.8.4 TransactionGuard

State and dependencies: Demo transaction repository, session decision repository and authenticated platform actor.

Operation	Contract
request_change(actor, new_destination) -> PendingAction	Create action_id and server-side digest of the canonical intended change; create a bound verification session.
authorize_and_execute(action_id, session_id, actor) -> ActionResult	In one database transaction, require ACTIVE account, matching user/action/digest, approved deployment policy, fresh ACCEPT and unused decision; apply update and consume authorization.

Behavior and invariants: Returning-user verification uses the active camera-derived template. Same result cannot authorize another action or a changed payout destination. Repeated execution with the same idempotency key returns the prior result. Client-visible ACCEPT text is never accepted as authorization.

3.9 Persistence and Data Lifecycle

3.9.1 Persistence Entities and Integrity Constraints

Entity	Keys and essential fields
demo.users	PK id; unique scoped user_ref; status; created_at; row_version.
demo.documents	PK id; FK user_id; encrypted_object_ref; consent_id; expires_at; state.
demo.enrollments	PK id; FK user_id, document_id, session_id, candidate_id; state; row_version.
demo.reviews	PK id; FK enrollment_id; reviewer_id; verdict; reason; snapshot_id; created_at.
engine.templates	PK id; FK/scoped user_ref; model_version; ciphertext; key_id; consent_id; status; activated_at; expires_at.
engine.sessions	PK id; user_ref; action_id; action_digest; intent; mode; state; nonce_hash; policy_version; expires_at; row_version.
engine.evidence	PK id; FK session_id; channel; status; allowed scalar JSON; model/config versions. Unique session_id/channel for final snapshot.
engine.decisions	PK id; unique FK session_id; snapshot_id; result; reason_codes; trace; created_at; consumed_at.
demo.transactions	PK action_id; user_id; destination_digest; status; unique consumed decision reference; idempotency key.
engine.audit_events	PK event_id; actor_ref; action; session_id; allowed metadata; time; monotonically ordered event sequence.

Foreign keys enforce ownership. Row versions and transaction locks protect activation and one-time authorization. Index ownership, time and review state; use UTC timestamps and versioned migrations. Validate evidence JSON before persistence.

3.9.2 Initial Retention Profile For The Controlled Demo

Data	Lifecycle
Ordinary frames and RGB traces	In memory only; release at completion, failure, cancel or expiry. Disable payload logging and application capture dumps.
Pending document and review still	Encrypted temporary objects for at most seven days; delete within 24 hours after closure or expiry. Expired cases require new capture.
Candidate template	Encrypted and inactive during review; delete on rejection, expiry or withdrawal.
Active template	Keep while the demo account and consent are active; revoke immediately on withdrawal and schedule deletion within 24 hours.
Audit metadata	Initial 30-day demo retention; exclude raw biometric imagery and routine BPM. Restricted research scalar evidence uses its own consent and retention.
Research corpus	Separate access-controlled storage; retention deadline recorded in each approved collection manifest, never an indefinite default.

These demo settings do not establish legal compliance. Use authenticated encryption with externally managed, rotatable keys and scoped decryption. Deletion includes derived artifacts and the documented backup lifecycle. Public packages exclude biometric data.

3.10 REST Interfaces

The routes listed below all require the proper actor scope and HTTPS. Authoritative identity, intent, and policy checks are owned by the API. Regular clients are unable to switch to an experimental mode or replace a reference. Inconsistent route examples in the V2 code are replaced by the canonical routes listed below.

3.10.1 Canonical REST Endpoints

Method and path	Input / successful response
POST /v1/sessions	intent, action_id, consent_id / 201 session_id, expires_at, capture_profile, scoped session credential.
POST /v1/sessions/{id}/finalize	Idempotency-Key / 202 job state and result URL; existing result returned consistently.
GET /v1/sessions/{id}/result	Authorized owner / 200 lifecycle state, nullable decision, safe reasons and next_action.
GET /v1/sessions/{id}/report	Authorized operator/researcher / 200 role-filtered evidence and trace.
POST /v1/sessions/{id}/cancel	Owner / 200 terminal state; idempotent.
POST /demo/enrollments	Consent and profile data / 201 enrollment_id and state.
POST /demo/enrollments/{id}/document	Multipart image / 201 document_id, quality summary or recapture reason.
POST /demo/enrollments/{id}/session	Owner / 201 enrollment-purpose session ticket bound to document.
POST /demo/reviews/{id}/decision	verdict, reason, expected_version / 200 new review/account state.
POST /demo/actions/payout-change	New destination and Idempotency-Key / 201 action_id, verification session.
POST /demo/actions/{id}/execute	session_id, Idempotency-Key / 200 executed or previously executed result.
POST /v1/sessions/{id}/ws-ticket	Owner / 201 one-time ticket valid for 30 seconds; only for that session socket.

3.10.2 Response and Error Contract

Schema_version, session_id, state, decision|null, reason_codes[], next_action, and policy_version are all included in the result response. A policy decision is not an HTTP authentication error; rather, it is an application outcome. Transport errors use {error:{code,message,retryable,request_id}}. 400 denotes incorrect input; 401/403 authentication/authorization failure; 404 unavailable authorized

resource; 409 state/idempotency conflict; 413 oversized input; 422 schema violation; 429 rate limit; 503 unavailable capacity or required model.

An Idempotency-Key is scoped to actor, route and request digest; reuse with different content returns 409. Unauthenticated users cannot inspect session existence. Operator reports require separate permissions. Uploaded documents have an initial 5 MiB limit and validated decoded dimensions; executable or unsupported content is rejected.

3.11 WebSocket and Web Client Interfaces

3.11.1 WebCaptureClient and EventGateway

State and dependencies: MediaStream, session ticket, bounded encoder queue and typed event dispatcher.

Operation	Contract
startVerification(ticket, callbacks) -> CaptureHandle	Request camera permission; show preview and guidance; start controlled capture. Browser permission is separate from project consent.
stop(reason) -> Promise<void>	Stop media tracks, release frames, close socket and request cancellation where needed.
onEvent(event) -> void	Validate schema/version and sequence; update the UI without locally changing the authoritative decision.
publish(event, audience) -> None	Server applies owner/role/mode filters before fan-out; never trusts client-selected event visibility.

Behavior and invariants: The browser must not calculate the authoritative acceptance result. It displays server state. Reconnecting a live media socket does not stitch an unqualified temporal window; interrupted capture requires a new attempt.

3.11.2 Canonical Channels and Events

WSS /v1/sessions/{id}/events accepts the one-time session ticket in the first message within five seconds. Before ticket validation, media packets are rejected. After validation the channel carries binary frames and typed control/events. No reusable access token is put in a URL. WSS /v1/admin/events uses an independently scoped one-time ticket and role filtering. Origin checking and ticket ownership are both enforced. [10, 14]

Event	Payload and audience
session.started	State, capture profile and server expiry / owner.
face.detected	Face count and positioning guidance / owner; geometry only in authorized test mode.
capture.quality	Usable flag and corrective reason / owner; detailed quality metrics for allowed roles.
rppg.progress	Valid duration and usable status / owner; waveform/BPM only in consented research view.
evidence.ready	Channel, status, allowed summary and model version / authorized audience.
fusion.decision	Result, safe reasons, next_action and policy version / owner.
session.completed	Terminal state and result URL / owner.
review.updated	Enrollment state and review reason / owner/reviewer.
audit.created	Sanitized operation and actor references / authorized operator.

Envelope: event_id, session_id, ts, seq, type, schema_version and payload. Event seq is server-generated per session; clients ignore duplicates and request current state when a gap is detected. Progress events may be coalesced; final decisions are persisted and recoverable by REST. Queue limits apply to both inbound frames and outbound metrics; a slow admin cannot block inference.

3.12 Security and Failure Handling

3.12.1 Roles and Permissions

Role	Permitted actions
User	Access own enrollment/session, grant or withdraw consent and request own action.
Enrollment reviewer	View only assigned review material; approve/reject/request recapture; cannot override failed biometric gates.
Session operator	Read sanitized session summaries and operational diagnostics; no default raw video or template access.
Researcher	Access authorized test sessions/corpus and experiment evidence under explicit research permission.
Service identity	Perform scoped inference, decrypt eligible references and persist decisions; not a browser credential.

3.12.2 Threat and Control Boundaries

There are distinct test cases for face mismatch, print, and replay. Physiological checks and visual PAD are complementary, but they are not statistically independent sensors and share an RGB source. Although they do not verify the actual camera origin, TLS, sequence checks, nonce binding, and expiration lessen the misuse of transport and sessions. Virtual sources are available with getUserMedia. Future work will focus on native source authentication and dynamic optical freshness. [8, 14]

Every session operation is verified by the server using actor ownership and mode. User, action, and service capacity are all subject to rate constraints. On transaction routes, the option to specify a research policy is disabled. When consent is withdrawn, reference use is revoked, incomplete relevant sessions are canceled, and data destruction is scheduled. Administrative actions are audited with clear justifications.

3.12.3 Failure Behavior

Failure	Required behavior
Camera denied or no device	Show camera guidance, cancel capture; no fabricated biometric decision.
No/multiple faces, unstable ROI	Reset eligible window and guide user; at capture deadline return RETRY if processing can conclude normally.
Low-quality rPPG	RETRY with guidance unless reliable mismatch/attack already supports REJECT.
Model exception or unavailable weights	FAILED / model error; no acceptance or fake zero score.
Database commit fails	Do not expose actionable acceptance; return processing/failure state and recover idempotently.
Network disconnect	Invalidate incomplete capture and clean resources; persisted completed result remains retrievable.
Repeated finalize/execute	Return the same result for the same request digest; conflicting content receives 409.

Required tests include cross-user access denial, altered action digest, double consumption, reviewer override attempts, stale candidate approval and test-mode leakage. The decision and its audit record commit atomically before any application authorization is consumed.

3.13 Evaluation and Metric Interfaces

3.13.1 ExperimentRunner

State and dependencies: Dataset manifest, frozen model profiles, split manifest and strategy registry.

Operation	Contract
extract(manifest, split) -> FeatureTable	Run identical channel extraction per sample; include provenance and status, but keep truth labels separate from model features.
evaluate(features, policies) -> RunReport	Run A, B, C, C-Q, D and L3 with immutable configurations; retain per-attempt outcomes and rule traces.

Operation	Contract
compare(runs, protocol) -> ComparisonReport	Use paired samples, declared working points, coverage and latency; do not retune using final test outcomes.

Behavior and invariants: Frames, windows and attack derivatives from one source recording remain in one split. Use subject-disjoint splits and a device holdout where data allows. Training fits models; validation selects thresholds/calibration; the test split is used once for final assessment.

3.13.2 Evaluation Configurations

Configuration	Purpose
A / Face only	Identity-only diagnostic control; not eligible for deployment authorization.
B / Face plus PAD	Estimate Visual PAD contribution; not eligible for deployment authorization.
C / L1 full channels	Fixed aggregate plus shared mandatory gates.
C-Q / quality only	Same gates and features, quality-weighted aggregate without extra L2 pattern rules.
D / L2	Same gates and features plus explicit quality/attack-aware policy.
L3	Same required gates and L2 safety holds; learned advisory score for remaining cases.
E / future challenge	Fresh challenge acquisition under a separate protocol; never fabricated from old recordings.

3.13.3 MetricCalculator

summarize(attempts, labels, protocol) -> MetricReport produces outcome counts for genuine and attack groups separately. For each group N includes initiated attempts: report ACCEPT/N, REJECT/N, RETRY/N and technical/expired/cancelled outcomes/N. Coverage=(ACCEPT+REJECT)/N. Also report genuine completion success and per-attack-species acceptance. Exclusions and denominators are explicit, not silently filtered.

PAD-layer APCER/BPCER are computed from the PAD assessment and applicable test protocol, with nonresponses reported separately; overall engine acceptance rates are not relabeled as pure PAD error rates. Face FMR/FNMR use separate 1:1 comparison labels. Risk among definitive decisions, coverage curves, p50/p95 acquisition-to-decision latency and technical failures accompany every policy comparison. Injection tests are separate. [9]

Retrying everyone has zero coverage, while rejecting everyone has excellent coverage but poor actual success. Only the policy cost is measured via offline evidence replay. End-to-end performance cannot be claimed using cached feature evaluation; full pipeline delay is determined independently of actual capture. Instead of treating associated frames as separate samples, uncertainty intervals should be resampled each subject or session.

3.14 Deployment and Performance Design

3.14.1 Deployment Architecture

Packages/bioauth-core, packages/bioauth-web, apps/api, apps/user-portal, apps/admin-console, experiments, deployments, and documents are all contained in a private monorepo. Web apps, PostgreSQL, bounded inference workers, and APIs are all run using Docker Compose. Consented tests and review materials are supported by a limited object store. The V2 scope is reflected in a separate update to the current public website.

3.14.2 Runtime Ownership and Limits

Each worker loads model artifacts from checksum-verified volumes once. While CPU/GPU-intensive analysis operates outside of the event loop, API workers handle I/O and authentication. Session_id, capture reference in owned memory, immutable policy, and model versions are all carried by a job. One concurrent capture per inference worker is allowed in the first deployment; further work is given a capacity response instead of an infinite queue. Only after memory/latency profiling does concurrency rise.

Each admitted session has a budget for decoded buffers, JPEG queues, and model memory. The worker is affected by cancelation, which ultimately prevents the release of capture resources. When a worker fails, the first process-local orchestration does not guaranty a smooth recovery of in-memory captures; instead, the session fails safely and a fresh effort is needed.

3.14.3 Performance Targets and Measurement

Requirement	Design treatment
30 FPS capture/processing target	Record requested, delivered, retained and processed FPS separately. Video input is not declared compliant solely because the browser requested 30 FPS.
3–5 second decision target	Preserve as an inherited target, not a demonstrated result. A five-second evidence window plus processing/network time can exceed it; profile and submit a justified revision to the advisor if required.

Requirement	Design treatment
100 ms per-frame upper target	Measure stage latency separately from sustained throughput. Serial 100 ms work cannot sustain 30 FPS; sampling and worker scheduling must be evaluated.
Bounded resources	Report CPU/GPU, peak memory, queue depth, dropped frames and timeout frequency on named hardware.
Reproducible installation	Lock dependencies, pin model manifests, provide migrations and a clean-machine Docker smoke test.

3.14.4 Release and Integration

Framework-independent APIs are exposed by Python Core, but camera and event handling are wrapped by the TypeScript SDK. After API stabilization and dependency/weight-license assessment, PyPI/npm is made public. Until ownership is confirmed, package names are working names. Source, documentation, and permitted artifacts are included in the release; biometric corpora and secrets are never included. Rather of being implicit first-version functionality, native Android, optical challenge, and deep-rPPG are still recorded backlog expansions.

3.15 Requirement Traceability and Verification

3.15.1 Requirement Traceability Matrix

Requirement	Design owner	Verification target
FR-01 / FR-02	Face detection / landmarks / CaptureBuffer	TC-FACE-01 single/multiple/no face; TC-LAND-01 stable coordinates.
FR-03	ROITracker	TC-ROI-01 bounds, pose change and track reset.
FR-04	CHROM/POS and SignalProcessor	TC-RPPG-01/02 known signals, filter band and timestamp gaps.
FR-05 / FR-06	SignalQualityAssessor / FrequencyAnalyzer	TC-RPPG-03/04 unusable signals, null BPM, method agreement.
FR-07	FaceMatcher / TemplateService	TC-MATCH-01 mismatch and version compatibility.

Requirement	Design owner	Verification target
FR-08	QualityAssessor	TC-QUALITY-01/02 low light, blur, movement and FPS.
FR-09	SafetyGate and policies	TC-FUSION-01 through 08 precedence, missing evidence, shared rPPG gate and C-Q/L2 comparison.
FR-10	Audit logger / MetricCalculator	TC-AUDIT-01 trace replay and denominator validation.
V2-FR-11	VisualPADService	TC-PAD-01/03 preprocessing, aggregation and qualified attack evidence.
V2-FR-12	ReliabilityAssessor / PatternAnalyzer	TC-FUSION-05/08 quality-only comparison and pattern holds.
V2-FR-13	TransactionGuard	TC-DEMO-01 digest binding, expiry and one-time execution.
V2-FR-14	EnrollmentService / ReviewService	TC-ENROLL-01/04 candidate provenance and prohibited override.
V2-FR-15	EventGateway / role filters	TC-DASH-01/03 owner isolation and test visibility.
V2-FR-16	Deferred light challenge	TC-CHAL-01/02 planned only after separate design approval.
V2-FR-17	Core / SDK / deployment	TC-PACK-01/03 clean install, schemas and permitted artifacts.

3.15.2 Version 1 Review and Implementation Gates

Approve the model manifest, threshold/calibration profile, transport qualification, and minimum usable-rPPG criteria prior to enabling operational acceptance. Until then, outside of clearly specified experimental runs, acceptance is still disabled. Maintain the same frozen profiles for all comparisons. The aforementioned identifiers will be expanded into data, methods, expected outcomes, and accountable roles in the test plan. [15]

Advisor review must resolve the web-first change against the historical Android commitment and the feasibility of inherited timing requirements. These items do not change the accepted mandatory-rPPG rule. The accompanying backlog identifies dependencies and planned work without claiming implementation completion or assigning unconfirmed team ownership.

4. Glossary

Term	Definition
Abstention	Withholding a definitive biometric verdict and requesting a new attempt or platform action.
Action binding	Association of a verification result with one user, one intended operation and an immutable operation digest.
Adapter	A boundary that translates one model/library interface into a stable project contract.
Bona fide	A genuine presentation in the evaluation protocol; not simply any image resembling the claimed user.
Calibration	A validation-based transformation or threshold selection that makes a score usable for a defined operating point.
Capture window	A time interval of eligible frames used for one physiological analysis.
Cosine similarity	Normalized vector dot product used to compare compatible face embeddings.
Evidence snapshot	Immutable set of channel outputs and versions supporting one policy evaluation.
Idempotency	Repeated equivalent requests produce the same effect without duplicate activation or execution.
Injection	Media supplied through a digital path that bypasses the assumed physical camera acquisition.
Model manifest	Versioned record of model artifact, preprocessing, labels, calibration and usage provenance.
Nonce	Random session value used in freshness and replay controls; not proof of physical presence.
Presentation attack	A physical presentation intended to interfere with biometric capture, such as print or screen replay.
Reliability	Suitability of a channel measurement under its observed acquisition conditions.
Replay	Re-use of recorded content or authorization; media replay and token reuse are different threat paths.

Term	Definition
Research mode	Explicitly authorized environment for richer evidence and controlled experiment configuration.
Strategy	An interchangeable implementation of a common algorithm or decision interface.
Subject-disjoint split	Training, validation and test partitions that do not share participants.
Template activation	Making a staged camera-derived reference eligible for later comparisons after successful review.
Temporal aggregation	Combining evidence from multiple frames into a session-level estimate.

5. References

[1] BioAuth Project Team. BioAuth Project Proposal. CMPE 491, Spring 2026.

[2] BioAuth Project Team. BioAuth Project Specifications. 27 March 2026.

[3] BioAuth Project Team. BioAuth Project Analysis Report. 6 April 2026.

[4] BioAuth Project Team. BioAuth Project High-Level Design Report. 13 May 2026.

[5] BioAuth Project Team. BioAuth V2 Technical Foundation, Product Architecture and Implementation Roadmap. Design Baseline v2.0, 12 October 2026.

[6] Object Management Group. Unified Modeling Language, Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

[7] B. Bruegge and A. H. Dutoit. Object-Oriented Software Engineering Using UML, Patterns, and Java. 2nd ed., Prentice Hall, 2004.

[8] J. Speth et al. Digital and Physical-World Attacks on Remote Pulse Detection. WACV, 2022, pp. 2407–2416. https://openaccess.thecvf.com/content/WACV2022/html/Speth_Digital_and_Physical-World_Attacks_on_Remote_Pulse_Detection_WACV_2022_paper.html

[9] ISO/IEC 30107-3:2023. Information technology — Biometric presentation attack detection — Part 3: Testing and reporting. https://www.iso.org/standard/79520.html

[10] FastAPI. WebSockets documentation. https://fastapi.tiangolo.com/advanced/websockets/

[11] G. de Haan and V. Jeanne. Robust Pulse Rate From Chrominance-Based rPPG. IEEE Transactions on Biomedical Engineering, 60(10), 2013. DOI: 10.1109/TBME.2013.2266196.

[12] W. Wang, A. C. den Brinker, S. Stuijk and G. de Haan. Algorithmic Principles of Remote PPG. IEEE Transactions on Biomedical Engineering, 64(7), 1479–1491, 2017. DOI: 10.1109/TBME.2016.2609282.

[13] SciPy. scipy.signal.sosfiltfilt reference. https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.sosfiltfilt.html

[14] MDN Web Docs. MediaDevices.getUserMedia. https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[15] TED University. CMPE 492 Senior Design Project II, 2026–2027 Fall Syllabus. User-provided Syllabus New.pdf, pp. 1–2 and 7.

[16] Minivision. Silent-Face-Anti-Spoofing source repository. https://github.com/minivision-ai/Silent-Face-Anti-Spoofing

[17] M. Savic and G. Zhao. OR-PAD official dataset repository and access conditions. https://github.com/marukosan93/OR-PAD

[18] ACM. Code of Ethics and Professional Conduct. 2018. https://www.acm.org/code-of-ethics

Web references consulted during design preparation in October 2026. External model weights and datasets require their own versioned usage records before incorporation. Source repositories do not by themselves establish redistribution rights for every artifact.